

Introduzione al C

Lezione 7

Sistemi Operativi - II Modulo (Canale A - L)

Prof. Paolo Zuliani

Dipartimento di Informatica

(Materiale didattico per cortesia del Prof. Emiliano Casalicchio)



SAPIENZA
UNIVERSITÀ DI ROMA



Agenda

- Funzioni
- Stringhe



Funzioni

- Ne abbiamo già vista una: `main`!

`<tipo di ritorno> nomefunzione (lista di parametri)`

- Lista di parametri può essere vuota
- Il tipo di ritorno può essere qualsiasi tipo del C, anche quelli definiti dal programmatore (che vedremo più avanti)
- Usate il tipo `void` per una funzione che non ritorna alcunché

Funzioni

Una funzione per la somma di due interi quadrati ($a^2 + b^2$)

```
int somma_quadrata (int a, int b) {  
    a *= a;  
    b *= b;  
    return a + b;  
}
```

Passaggio parametri
per valore.

- I parametri sono *variabili locali*!!
- Dove vengono allocate?

Ricordate la definizione generale:

<tipo di ritorno> nomefunzione (lista di parametri)



Funzioni

Il quadrato di un vettore di interi ($a[0]^2, a[1]^2, a[2]^2, \dots$)

In C **non si può** passare un array per valore.
(Molto inefficiente.)

```
void array_quadro (int a[], unsigned int n) {  
    for (int i = 0; i < n; i++)  
        a[i] *= a[i];  
}
```

La funzione non ritorna
alcunché.

Esercizio 1 (in classe)

Usate la funzione `array_quadro` su questo vettore di 5 double:

`{1.0, 4.2, -2.0, 2.0, 0.1}`

Funzioni

- Ricordate che l'identificatore di un array non è altro che un puntatore al primo elemento dell'array.
- Posso quindi scrivere equivalentemente:

```
void array_quadro (int *a, unsigned int n) {  
    for (int i = 0; i < n; i++)  
        a[i] *= a[i];  
}
```

Sintassi per il passaggio di
parametri per riferimento

Esercizio 2 (in classe)

Cambiate l'intestazione della funzione `array_quadro` dell'esercizio precedente e ricompilate.



Stringhe



Stringhe

- Array di caratteri ... ma un array speciale
 - Ogni elemento dell'array è un carattere della stringa
 - Ultimo elemento dell'array contiene carattere di fine stringa
 - ``Hello`` → {H, e, l, l, o, \0} dove \0 è detto NULL

H	e	l	l	o	\0
---	---	---	---	---	----

Stringhe: Inizializzazione

- `char s[10]="Lezione 7";`

s[0]	s[1]	s[2]	s[3]	s[4]	s[5]	s[6]	s[7]	s[8]	s[9]
L	e	z	i	o	n	e		7	\0

- `char r[10]="L7 3Apr";`

r[0]	r[1]	r[2]	r[3]	r[4]	r[5]	r[6]	r[7]	r[8]	r[9]
L	7		3	A	p	r	\0	?	?

- Carattere fine stringa aggiunto automaticamente
- `char r[10]={`L`,`7`,``,`3`,`A`,`p`,`r`};`
 - `\0`, non inserito automaticamente, quindi non è una stringa. **Sconsigliato**
 - `char r[10]={`L`,`7`,``,`3`,`A`,`p`,`r`,`\0`};` **OK**

Stringhe: Inizializzazione (cont.)

- `char r[10]='`L9 4apr`';`
 - `r[1]` è ``9``
 - `r[4]` è ``a``
 - `r[7]` è ``\0``
 - `r[8]` e `r[9]` sono invece *indeterminati*
- `char r[]='`L9 4apr`';`
 - OK, viene allocata una stringa di dimensione 8

Lunghezza, Copia, confronto, Concatenazione

- Nel file `stringhe.c` esempi di uso funzioni per
 - Calcolo lunghezza `[strlen(s)]`
 - Copia stringhe `[strcpy(s1, s2)]`
 - Confronto stringhe `[strcmp(s1, s2)]`
 - Concatenazione stringhe `[strcat(s1, s2)]`
- Leggete il man per vedere come funzionano esattamente.
- Attenti alla lunghezza delle stringhe per le operazione di copia e concatenazione!!

Altre funzioni

- `char *strncpy(char *dest, const char *src, size_t n);`
 - Al più n bytes di src sono copiati
- `char *strncat(char *dest, const char *src, size_t n);`
 - Al più n bytes di src sono concatenati
- `int strncmp(const char *s1, const char *s2, size_t n);`
 - Confronta i primi n bytes di s1 e s2

Esercizio 3 (in classe)

- Scrivere un programma che, date due stringhe `s1` e `s2` con `strlen(s1)>5` e `strlen(s2)>5`, copi i primi 5 caratteri di `s2` in `s1`. Usare la funzione `strncpy`
- Scrivere un programma che date 2 stringhe `s1` e `s2` concateni i primi `n` caratteri di `s1` ad `s2`. Usare la funzione `strncat`
- Scrivere un programma che date 2 stringhe `s1` e `s2` determini quella di dimensione minore, denotata come `nMin`, e confronti i primi `nMin` caratteri di `s1` e `s2`.

Esercizio 4

- Progettare ed implementare una funzione che abbia lo stesso comportamento di

```
char *strncpy(char *dest, const char *src,  
size_t n);
```

- Per casa

Esercizio 5

- Progettare ed implementare una funzione che abbia lo stesso comportamento di

```
char *strncat(char *dest, const char *src,  
size_t n);
```

- Per casa



Input / output

- `printf` usando `%s` come segnaposto
- ma vediamo funzioni più flessibili
 - Output
 - `int putchar(int c);`
 - `int puts(const char *s);`
 - `int fputs(const char *s, FILE *stream);`
 - Input
 - `char *gets(char *s);` **DEPRECATED (obsoleta)**
 - `char *fgets(char *s, int size, FILE *stream);`
 - `int getchar(void);`

Nota su `putchar()` e `getchar()` (I)

```
int getchar(void)
```

```
int putchar(int c)           //perché c non è char?
```

Queste funzioni ritornano `int` per gestire eventuali errori di input/output tramite la costante `EOF` (di solito vale **-1**):

- `getchar()` ritorna `EOF` quando l'input da `stdin` termina (ad es. premendo Ctrl-D), altrimenti ritorna il carattere letto;
- `putchar(c)` ritorna `EOF` se si verifica un errore di scrittura su `stdout`, altrimenti ritorna `c`.

Nota su `putchar()` e `getchar()` (II)

- Di norma funzionano in modalità *buffered* («tamponata»)
 - A basso livello, **NON** fanno input/output di un singolo carattere
 - I caratteri sono inseriti in un *buffer* ('memoria tampone') che viene svuotato quando pieno o quando lo segnala il programmatore
- Ad esempio, `getchar()` ritorna al chiamante solo quando:
 - incontra un Ctrl-D e il buffer è vuoto; oppure
 - incontra '\n' e il buffer **non** è vuoto

Esercizio 6

- Scrivere un programma che, facendo uso della funzione `getchar()` legga dallo `stdin` una stringa di lunghezza arbitraria (ovvero lunghezza non nota a priori)

Esempio (getch.c in Lezione7.tgz)

```
#include <stdio.h>
#define MAX 10
void main() {
    int nc = -1;
    char str[MAX+1];    // un char in più per '\0'

    // lettura ('bufferizzata')
    // scrive direttamente nella stringa
    while ((str[++nc] = getch()) != EOF) ;

    // vediamo cosa c'è nella stringa
    for (int i = 0; i < MAX+1; i++)
        printf("carattere = %d\n", str[i]);

    // scrive '\0' a fine stringa
    str[(nc > MAX) ? MAX : nc] = '\0';

    printf("nc = %d; str = %s\n", nc, str);}
```

Cosa succede
se EOF < -128??

Una soluzione più portabile: funziona su Mac e Linux

```
#include <stdio.h>
#define MAX 10
void main() {
    char str[MAX+1];    // un char in più per '\0'

    int c, nc = 0;
    // getch() ritorna un int!
    while ((c = getch()) != EOF)
        // memorizza c solo se diverso da EOF
        str[nc++] = c;

    // vediamo cosa c'è nella stringa
    for (int i = 0; i < MAX+1; i++)
        printf("carattere = %d\n", str[i]);

    // scrive '\0' a fine stringa
    str[(nc > MAX) ? MAX : nc] = '\0';

    printf("nc = %d; str = %s\n", nc, str);}
```



Domande?

